

1. Định thức và các tính chất

Bài làm và kết quả:

```
import numpy as np
n = 4
X = np.array(range(1,n+1))
sigma = np.array([4,3,2,1])
def sgn_by_def(sigma):
    ketqua = 1.0
    for i in range(len(X)-1):
        for j in range(i+1,len(X)):
            ketqua = ketqua * ((X[i]-X[j])/(sigma[i]-sigma[j]))
    return int(ketqua)

sigma = np.array([2,1,3,4])
sgn_by_def(sigma)

✓ 0.9s
```

-1

Bài làm và kết quả:

```
sigma = np.array([1,2,3,4])
sgn_by_def(sigma)

✓ 0.0s
```

1

Bài làm và kết quả:

```
sigma = np.array([4,3,2,1])
sgn_by_def(sigma)

✓ 0.0s
```

1

Bài làm và kết quả:

```
from itertools import permutations
n = 3
X = []
for i in range (1, n+1):
    X.append(i)
Sn = list(permutations(X)) # tạo hoán vị của tập X
print (Sn)
```

✓ 0.0s

```
[(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1)]
```

Bài làm và kết quả:

```
Sn[3].index(2) # sẽ là 0 vì 2 ở là vị trí đầu tiên của Sn[3].
det = 0 # bước này cố duy nhất 1 lệnh, cố ý nghĩa khởi tạo giá trị ban đầu của định thức.
import numpy as np
```

```
def phatsinh_dinhthuc(n):
    X = []
    for i in range(1, n+1):
        X.append(i)

    Sn = list(permutations(X))
    dinhthuc = ""

    for sn in Sn:
        sigma = np.array([1])
        sigma.resize([n])
        product = ""

        for i in range(1, n+1):
            sigma[sn.index(i)] = i
            product = product + "a" + str(i) + str(sn.index(i) + 1)

        dau = sgn_by_def(sigma)

        if (dau != 1):
            product = " - " + product
        else:
            product = " + " + product

        dinhthuc = dinhthuc + product

    return dinhthuc
```

```
# sgn_by_def(sigma) là gọi hàm bên phía trên đã xây dựng
phatsinh_dinhthuc(2)
```

```

-----
IndexError                                Traceback (most recent call last)
Cell In[16], line 34
     31     return dinhthuc
     33 # sgn_by_def(sigma) là gọi hàm bên phía trên đã xây dựng
--> 34 phatsinh_dinhthuc(2)

Cell In[16], line 22
     19     sigma[sn.index(i)] = i
     20     product = product + "a" + str(i) + str(sn.index(i) + 1)
--> 22 dau = sgn_by_def(sigma)
     24 if (dau != 1):
     25     product = " - " + product

Cell In[1], line 9
      7 for i in range(len(X)-1):
      8     for j in range(i+1,len(X)):
--> 9         ketqua = ketqua * ((X[i]-X[j])/(sigma[i]-sigma[j]))
     10 return int(ketqua)

IndexError: index 2 is out of bounds for axis 0 with size 2

```

n = 2 lỗi vì độ dài sigma và X không đồng bộ.

```

Sn[3].index(2) # sẽ là 0 vì 2 ở là vị trí đầu tiên của Sn[3].
det = 0 # bước này có duy nhất 1 lệnh, có ý nghĩa khởi tạo giá trị ban đầu của định thức.
import numpy as np

def phatsinh_dinhthuc(n):
    X = []
    for i in range(1, n+1):
        X.append(i)

    Sn = list(permutations(X))
    dinhthuc = ""

    for sn in Sn:
        sigma = np.array([1])
        sigma.resize([n])
        product = ""

        for i in range(1, n+1):
            sigma[sn.index(i)] = i
            product = product + "a" + str(i) + str(sn.index(i) + 1)

        dau = sgn_by_def(sigma)

        if (dau != 1):
            product = " - " + product
        else:
            product = " + " + product

        dinhthuc = dinhthuc + product

    return dinhthuc

# sgn_by_def(sigma) là gọi hàm bên phía trên đã xây dựng
phatsinh_dinhthuc(3)
✓ 0.0s

' + a11a22a33 - a11a23a32 - a12a21a33 + a13a21a32 + a12a23a31 - a13a22a31 '

```

n = 3 chạy được là do trùng kích thước mảng

Bài làm và kết quả:

```

def tinhtoan_dinhthuc(A):
    X = []
    import math
    n = int(math.sqrt(A.size)) # <-- bổ sung để sử dụng sqrt
                                # <-- với ma trận vuông, kích thước là căn số
    for i in range(1, n+1):
        X.append(i)

    Sn = list(permutations(X))
    dinhthuc = 0 # ban đầu giá trị định thức là 0

    for sn in Sn:
        sigma = np.array([1])
        sigma.resize([n])
        product = 1 # đặt giá trị cho tích ban đầu là 1

        for i in range(1, n+1):
            sigma[sn.index(i)] = i
            product = product * A[i-1][sn.index(i)] # lưu ý chỉ số
            # print(A[i-1][sn.index(i)])

        dau = sgn_by_def(sigma)

        if (dau != 1): # không xét trường hợp dấu = 1
            product = product * (-1) # khi dấu = -1 thì nhân với -1

        dinhthuc = dinhthuc + product

    return dinhthuc

matran = np.array([ [3,5,-8], [4, 12, -1], [2,5,3] ])
tinhtoan_dinhthuc(matran) # MT A

```

✓ 0.0s

np.int64(85)

2. Định thức và ma trận khả nghịch

3. Quy tắc Cramer

Bài làm và kết quả:

```
import numpy as np
A = np.array([[4, -2],[3, -5]]) # khai báo ma trận A
A1 = np.array([[10, -2],[11, -5]])
A2 = np.array([[4, 10],[3, 11]])
from scipy import linalg # lưu ý: hàm tính định thức của ma trận là scipy.linalg.det(A)
detA = linalg.det(A) # tính định thức cho ma trận A
detA1 = linalg.det(A1)
detA2 = linalg.det(A2)
print (detA, detA1, detA2)
```

✓ 2.7s

-14.0 -28.0 14.000000000000004

Bài làm và kết quả:

```
if (detA != 0):
    x1 = detA1 / detA
    x2 = detA2 / detA
    print ("Hai nghiệm của phương trình là: ", x1, x2)
```

✓ 0.0s

Hai nghiệm của phương trình là: 2.0 -1.0000000000000002

Bài làm và kết quả:

```
import numpy as np
from itertools import permutations
```

```
print(tinh_toan_dinh_thuc(A))
print(tinh_toan_dinh_thuc(A1))
print(tinh_toan_dinh_thuc(A2))
```

✓ 0.0s

-14

-28

14

Bài làm và kết quả:

```
# Sử dụng hàm det() của lớp linalg của thư viện scipy
import numpy as np
A = np.array([
    [-1, 2, -3],
    [ 2, -2, 1],
    [ 3, -4, 4]
])
B = np.array([1, 3, 2])
A1 = np.array([
    [ 1, 2, -3],
    [ 3, -2, 1],
    [ 2, -4, 4]
])
A2 = np.array([
    [-1, 1, -3],
    [ 2, 3, 1],
    [ 3, 2, 4]
])
A3 = np.array([
    [-1, 2, 1],
    [ 2, -2, 3],
    [ 3, -4, 2]
])
from scipy import linalg
detA = linalg.det(A)
detA1 = linalg.det(A1)
detA2 = linalg.det(A2)
detA3 = linalg.det(A3)
print(detA, detA1, detA2, detA3)
x = detA1 / detA
y = detA2 / detA
z = detA3 / detA
print(x, y, z)
```

✓ 0.0s

0.0 0.0 0.0 0.0

Bài làm và kết quả:

```
● # Sử dụng hàm tinhtoan_dinhthuc() viết bên trên

detA = tinhtoan_dinhthuc(A)
detA1 = tinhtoan_dinhthuc(A1)
detA2 = tinhtoan_dinhthuc(A2)
detA3 = tinhtoan_dinhthuc(A3)

print(detA, detA1, detA2, detA3)

✓ 0.0s

0 0 0 0
```

Bài làm và kết quả:

```

import numpy as np
from scipy import linalg
# Ma trận hệ số
A = np.array([
    [-1, 2, -3],
    [ 2, -2, 1],
    [ 3, -4, 4]
])
# Vector hằng số
B = np.array([1, 3, 2])
# Các ma trận theo quy tắc Cramer
Ax = np.array([
    [1, 2, -3],
    [3, -2, 1],
    [2, -4, 4]
])
Ay = np.array([
    [-1, 1, -3],
    [ 2, 3, 1],
    [ 3, 2, 4]
])
Az = np.array([
    [-1, 2, 1],
    [ 2, -2, 3],
    [ 3, -4, 2]
])
# Tính định thức
detA = linalg.det(A)
detAx = linalg.det(Ax)
detAy = linalg.det(Ay)
detAz = linalg.det(Az)
# Kiểm tra điều kiện có nghiệm duy nhất
if abs(detA) < 1e-10:
    print("Hệ phương trình không có nghiệm duy nhất (detA = 0)")
else:
    x = detAx / detA
    y = detAy / detA
    z = detAz / detA

```

```
y = detAy / detA
z = detAz / detA
print("Nghiem của hệ phương trình:")
print("x =", x)
print("y =", y)
print("z =", z)
x, y, z = detAx/detA, detAy/detA, detAz/detA # tính toán x, y, z
print ("Nghiem của phương trình: ", "x=", x, "y=", y, "z = ", z)
```

✓ 0.0s

Hệ phương trình không có nghiệm duy nhất (detA = 0)
Nghiem của phương trình: x= nan y= nan z = nan
C:\Users\Tran Thinh\AppData\Local\Temp\ipykernel_25304\3536226587.py:51: RuntimeWarning: invalid value encountered in scalar divide
x, y, z = detAx/detA, detAy/detA, detAz/detA # tính toán x, y, z

Bài làm và kết quả:

```
import numpy as np
# Ma trận hệ số
A = np.array([
    [-1, 2, -3],
    [ 2, -2, 1],
    [ 3, -4, 4]
])
# Vector hằng số
B = np.array([1, 3, 2])
# Ma trận Cramer
AX = np.array([
    [1, 2, -3],
    [3, -2, 1],
    [2, -4, 4]
])
AY = np.array([
    [-1, 1, -3],
    [ 2, 3, 1],
    [ 3, 2, 4]
])
AZ = np.array([
    [-1, 2, 1],
    [ 2, -2, 3],
    [ 3, -4, 2]
])
# Tính định thức bằng hàm đã viết
det = tinhtoan_dinhthuc(A)
detX = tinhtoan_dinhthuc(AX)
detY = tinhtoan_dinhthuc(AY)
detZ = tinhtoan_dinhthuc(AZ)
# Tính nghiệm
x, y, z = detX/det, detY/det, detZ/det
print("Nghiem", "x =", x, "y =", y, "z =", z)
```

✓ 0.0s

Nghiem x = nan y = nan z = nan
C:\Users\Tran Thinh\AppData\Local\Temp\ipykernel_25304\78222266.py:39: RuntimeWarning: invalid value encountered in scalar divide
x, y, z = detX/det, detY/det, detZ/det

4. Bài toán ứng dụng 1: Tính diện tích đa giác, thể tích và các phương trình đường, mặt.

Bài làm và kết quả:

```
import sympy as sp
TG = sp.Matrix([[1, 0, 1],[4, 3, 1], [2, 2, 1]])
1/2*TG.det()
print(1/2*TG.det())
```

✓ 0.0s

1.5000000000000000

Bài làm và kết quả:

```
from sympy import Matrix

M = Matrix([[0, 4, 1, 1], [4, 0, 0, 1], [3, 5, 2, 1], [2, 2, 5, 1]])
print(M.det())
print(1/6*M.det())
```

✓ 0.0s

-72

-12.000000000000000

Bài làm và kết quả:

```
from sympy import * # lúc này ta sử dụng trực tiếp thư viện
x, y, z = symbols('x y z')
MP = Matrix([[x, y, z, 1],[-1, 3, 2, 1],[0, 1, 0, 1],[-2, 0, 1, 1]])
print(MP.det())
```

✓ 0.1s

$-4x + 3y - 5z - 3$

Bài tập chương 4:

Bài làm và kết quả:

```

# Câu 1:
import numpy as np
from scipy import linalg
# Tính ma trận hệ số kép
def cofactor_matrix(A):
    n = A.shape[0]
    C = np.zeros((n, n))

    for i in range(n):
        for j in range(n):
            # Ma trận con M_ij
            M = np.delete(np.delete(A, i, axis=0), j, axis=1)
            C[i, j] = ((-1)**(i + j)) * linalg.det(M)
    return C

# Tính ma trận liên hợp
def adjoint_matrix(A):
    return cofactor_matrix(A).T

# Ví dụ minh họa
A = np.array([
    [1, 2, 3],
    [0, 4, 5],
    [1, 0, 6]
])
C = cofactor_matrix(A)
Adj = adjoint_matrix(A)
print("Ma trận A:")
print(A)
print("\nMa trận hệ số kép:")
print(C)
print("\nMa trận liên hợp:")
print(Adj)

```

Ma trận A:

```
[[1 2 3]
 [0 4 5]
 [1 0 6]]
```

Ma trận hệ số kép:

```
[[ 24.   5.  -4.]
 [-12.   3.   2.]
 [ -2.  -5.   4.]]
```

Ma trận liên hợp:

```
[[ 24. -12.  -2.]
 [  5.   3.  -5.]
 [ -4.   2.   4.]]
```

Bài làm và kết quả:

```

# Câu 2:
import sympy as sp
# Khai báo biến
x, y = sp.symbols('x y')
# Ba điểm đã biết
x1, y1 = 1, 2
x2, y2 = 4, 6
x3, y3 = 5, 2
# Lập ma trận định thức
M = sp.Matrix([
    [x**2 + y**2, x, y, 1],
    [x1**2 + y1**2, x1, y1, 1],
    [x2**2 + y2**2, x2, y2, 1],
    [x3**2 + y3**2, x3, y3, 1]
])
# Tính định thức
circle_eq = sp.expand(M.det())
print("Phương trình đường tròn:")
print(circle_eq, "= 0")

```

✓ 0.1s

Phương trình đường tròn:

$$-16x^2 + 96x - 16y^2 + 116y - 248 = 0$$